#5 We take a peek at the queue produced by the call at line #4…

#6 …if its top element is still at an infinite distance, we need to remove it, because it means we have added less than `n` elements to the queue.

#7 Once that’s taken care of, we can just return to the caller the queue with the elements we have found.

#8 Internal version of the function taking an already initialized priority queue. The queue will encapsulate the logic about keeping track of the n-th closest neighbors and their distances, which we’ll use to prune search.

#9 If node is `null`, we are traversing an empty tree, so the nearest neighbors can’t change with respect to what we have already found.

#10 Once we are sure current node is not null, we can compute the distance between current node’s point and target.

#11 We try to insert the tuple (current distance, current point) into the bounded (max) priority queue. This helper data structure takes care of keeping only the smallest `n` tuples, so that current point will only be added if it its distance is among the `n` smallest distances found so far.

#12/#13 Check if the target point is on the left branch of the split. If it is, the left branch is the closest to the target point, otherwise it is the furthest.

#14 We certainly need to traverse the closest branch in search of the nearest neighbor. It is important to do so first and update the priority queue, and so the distance to the `n`-th closest neighbor, to improve pruning.

#15 We need to retrieve the distance to the `n`-th closest neighbor, and we can do so by peaking at the tuple at the top of the bounded priority queue. Notice that this works